



Frontend Testing

Unit & Component

SPRING 2026

OVERVIEW

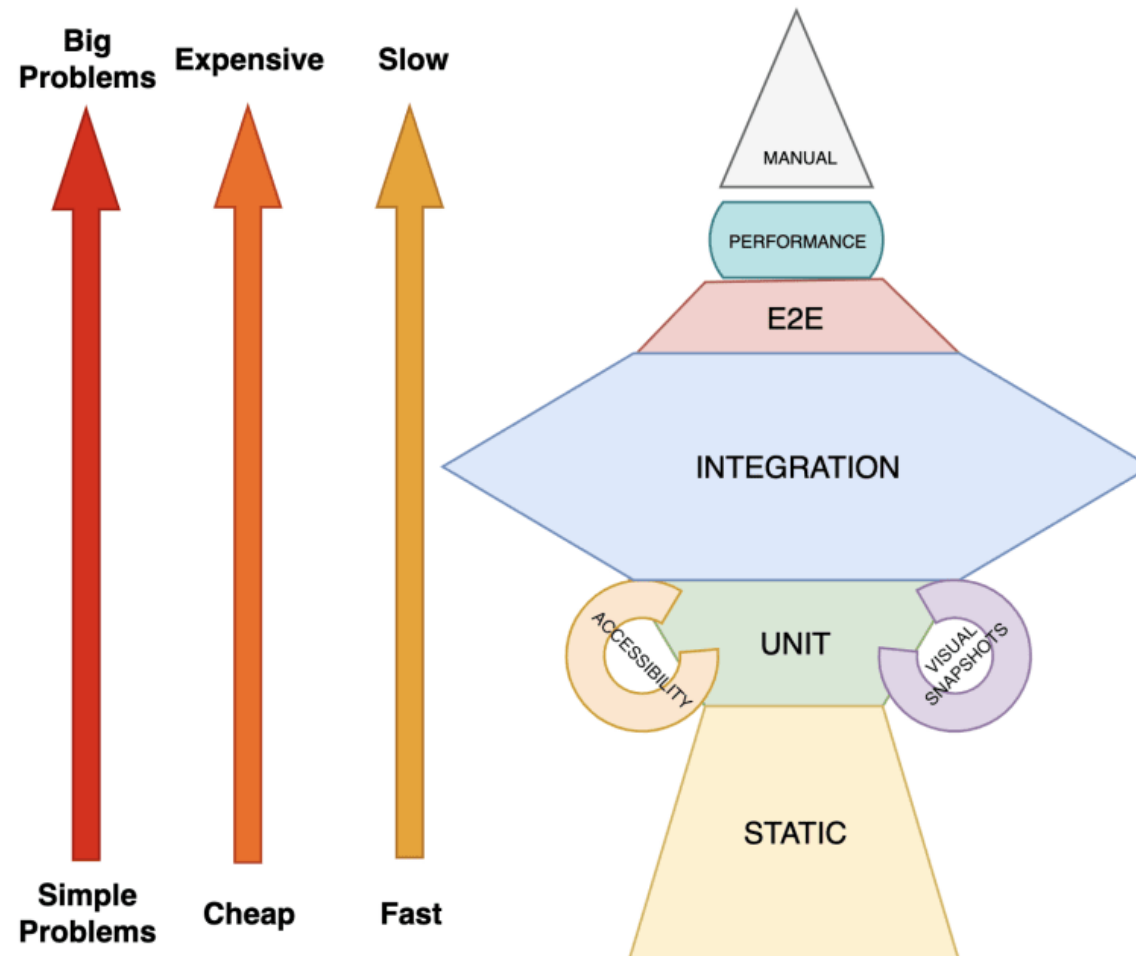
- Testing on the Frontend
- Unit & Component Testing
- Writing Unit Tests
- Use Case
- Tools
- Handling Dependencies
- Automation
- Best Practices

Testing on the Frontend

Types of Tests

Static Tests	Performed without requiring the code to be executed.	Code formatters (Prettier) Linters (ESLint) Static type checking (TypeScript)
Manual Tests	Tests performed without using any automated tools to reproduce exactly what the end user will do.	
Unit Tests	Used to verify that the smallest building blocks, tested in isolation , work as expected. Isolated components are units. => Unit testing = Component testing	
Integration Tests	Used to verify that two or more components/units work together in harmony. They test one system's behavior (Frontend).	
End-to-end Tests	Used to automate the browser and try to reproduce a typical user flow around the application covering all the systems (Frontend, Backend) and their interactions.	
Others	Accessibility – Performance – Visual Snapshots	

The Testing Trophy



Unit & Component Testing

Advantages

Used to verify that the smallest building blocks, tested in **isolation**, work as expected.



Maintainability & Reliability

Provide a safety net for refactoring.



Regression & Bugs

Help detect and fix issues early in the development process.



Improved Code Quality

Encourage developers to create modular, loosely coupled code.



Documentation

Serves as a documentation of how each unit of code is intended to function.

Writing Unit Tests

Techniques

Test-driven Development (TDD)

Tests are written before coding:

1. A test is written that fails since the implementation hasn't been written.
2. The simplest implementation is written to pass the test.
3. The code is refactored and improved, as necessary.

These steps are repeated until the feature is completed.

Test-later Development (TLD)

Tests are written after the feature is implemented.

Mostly used when technical details are unknown and the functionality is to be determined as the code is written.



When encountering a bug in an uncovered scenario, a test should be written to ensure the bug is caught in the future if it resurfaces. It is advisable to write a test before fixing it (use TDD).

Structure

The Arrange, Act and Assert (AAA) Pattern

A common structure used in writing unit tests, it helps organize the test code into three main sections:

Arrange – the setup phase – usually involves creating objects, setting up the environment, and putting the system into a known state.

Act – the action or operation that you want to test - usually involves calling a method or performing some operation on the system.

Assert – the verification phase – compare the result of the action against your expectation.

Syntax

it function

Describes a test

Takes a test's title as a string and the test as a function

```
it("is a test", () => {});
```

describe function

Describes a Suite: a

group of tests
Takes the collection's title as a string and test grouping as a function

```
describe("A group of tests", () => {  
  it("is a test", () => {});  
  it("is a second related test", () => {});  
});
```

expectations & matchers

expect function that takes an actual value and is chained with a matcher
The matcher takes the expected value

```
it("has a positive case",  
  () => {  
    expect(true).toBe(true);  
  });
```

beforeEach

beforeEach function runs some shared setup before each of the tests in the describe in which it is called.

```
beforeEach( () => {  
  /* Setup for the component  
  used in all tests */  
});
```

Useful Links

- [Jasmine Documentation](#)
- [List of matchers](#)
- [Creating custom matchers](#)

describe and it blocks are functions

- they can contain any executable code necessary to implement the tests.
- JavaScript scoping rules apply: variables declared in a describe are available to any block inside the suite



A test that is not given a reason to fail, will always pass.

Angular

Angular already has the **Jasmine** testing framework and the **Karma** test runner setup for writing unit tests.

Testing features provided by and specific to Angular:

Useful Links

- [Angular Testing Guide](#)

TestBed

create a test module that contains the components, services and directives needed to test a specific feature.

ComponentFixture

a wrapper around a component instance and its associated template. It provides methods to interact with the component and its DOM elements during testing.

inject() & TestBed.get()

functions used to access services and dependencies in your test cases.



TestBed.createComponent() does not trigger change detection by default. **detectChanges()** should be called to perform data binding. This behavior is intentional to be able to set up different testing values.

Use Case

User Story Description

As a user I want to have a button that lets me modify the quantity of an item in my cart.



Context

Every product card should have this button. The button is hidden for non-logged in users.

Technical Requirements

A request should be sent on every quantity change.

Acceptance Criteria - Best Practices

The Goal

Explain the conditions a specific user story must satisfy to be accepted by a user, a customer or other systems.

Conditions

- Should describe what the end result should be, not the process of achieving it.
- Must be defined before the development team starts working on a specific user story.

The Format - Given/When/Then Template

Given some precondition

Arrange

When I do some action

Act

Then I expect some result

Assert

Acceptance Criteria - Example

Acceptance Criteria

1. *Given i am a logged in user*
When i reach the maximum number of items
Then i expect a toast message to be displayed
and the button to be disabled
1. *Given I am not a logged in user*
When I view a product
Then i shouldn't see the add to cart
1. *Given I am a logged in user and i haven't selected an item*
When I click on the '+' button
Then I expect to see an expanded button with the count
and the '-' button visible
1. *Given I am a logged in user and i have already an item to the cart*
When I click on the '-' button
Then i expect the quantity to decrease
4. *Given I m a logged in user and I have one item in the cart (the quantity is 1)*
When I click the '-' button
Then i expect the product to be removed from the cart //
then i expect only the '+' button to be shown
5. *Given Im a logged in user and i have at least one item in the cart*
When i click on the '+' button
Then i expect the quantity to increase (be incremented)
6. *Given I'm a logged in user*
When I click on the '+' button
Then a POST request should be sent to the BE
7. *Given I'm logged in user*
When i click on an item that is no longer available
Then i expect a toast message to appear and the button to be disabled

User-focused Testing - Best Practices

- Tests should avoid including implementation details. Implementation details are things which users of your code will not typically use, see, or even know about.
- Tests should be maintainable so refactors (changes to implementation but not functionality) don't break the tests.
- Tests should be written in a way that closely resemble how your web pages are used.



```
it("setOpenIndex sets the open index state properly", () => {})
```



```
it ("can open accordion items to see the contents", () => {})
```

Tools

Tools -Testing-library

- A solution that provides light utility functions in a way that encourages testing practices and makes it very difficult to include implementation details in my tests.
- The main utilities it provides involve querying for nodes similarly to how users would find them
- `npm install --save-dev @testing-library/angular`

Useful Links

- [API](#)
- [Examples](#)
- [User Interactions Guide](#)

Queries	Methods to find elements on the page Single elements: <code>getBy</code> , <code>findBy</code> , <code>queryBy</code> Multiple elements: <code>getAllBy</code> , <code>findAllBy</code> , <code>queryAllBy</code>	<code>get</code> throws error <code>query</code> returns null <code>find</code> returns Promise = <code>get</code> + <code>waitFor</code>
User Interactions	a companion library for Testing Library that simulates user interactions by dispatching the events that would happen if the interaction took place in a browser.	<pre>const user = userEvent.setup(); await user.click(element) await user.type(element, ' World!')</pre>

Tools -Testing-library

```
it( expectation: 'should increment the counter', assertion: () :void => {  
  const fixture : ComponentFixture<QttyButtonCom... = TestBed.createComponent(QttyButtonComponent);  
  fixture.detectChanges();  
  const component : QttyButtonComponent = fixture.componentInstance;  
  component.increment();  
  expect(component.quantity).toBe( expected: 1);  
});
```

VS.

```
import { render, fireEvent } from '@testing-library/angular';  
import {QttyButtonComponent} from "../qtty-button.component";  
  
it( expectation: 'should increment the counter', assertion: async () : Promise<void> => {  
  const { getByText } = await render(QttyButtonComponent);  
  fireEvent.click(getByText( args: 'Increment'));  
  expect(getByText( args: '1')).toBeTruthy();  
});
```

Useful Links

- [API](#)
- [Examples](#)
- [User Interactions Guide](#)

Handling Dependencies

Mocking

Mocking is a process used when the unit being tested has external dependencies.

Involves replacing real dependencies with objects (mocks) to control their behavior and isolate the unit being tested.

Mock calls to external APIs to make sure tests don't make actual network requests.

Over-mocking can result in tests that do not accurately reflect real-world interactions.

```
class MockUserService {  
  isLoggedIn = true;  
  user = { name: 'Test User' };  
}  
  
TestBed.configureTestingModule({  
  providers: [{  
    provide: UserService,  
    useClass: MockUserService  
  }]  
});
```

```
const twainService = jasmine.createSpyObj('TwainService', ['getQuote']);  
  
getQuoteSpy = twainService.getQuote.and.returnValue(of(testQuote));  
  
TestBed.configureTestingModule({  
  imports: [TwainComponent],  
  providers: [{ provide: TwainService, useValue: twainService }],  
});
```

Tests Characteristics

Isolated – tests should return the same results regardless of the order in which they are run.

Fast – tests should run quickly.

Behavioral – tests should be sensitive to changes in the behavior of the code under test. If the behavior changes, the test result should change.

Structure-insensitive – tests should not change their result if the structure of the code changes.

Specific – if a test fails, the cause of the failure should be obvious.

Predictive – if the tests all pass, then the code under test should be suitable for production.

Automated – tests should run without human intervention.

Automation

Automating tests – CI/CD



Next steps

Moving forward

- Static tests should be included in all projects.
 - ESLint, Prettier formatting on save, and TypeScript.
- Add unit tests to the add to cart functionality in your projects using testing-library

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld.
757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA

